

Codes et réductions — guide d'intégration mobile

Un seul champ à l'achat, trois usages derrière. Toutes les réponses ci-dessous sont issues d'**appels réels** sur l'environnement de développement.

Référence : issue #247. Remplace la section « code » du [guide parrainage](#).

1. La règle, en une phrase

L'acheteur saisit UN code. Le serveur décide de son effet : une réduction sur le prix, une commission au propriétaire du code, ou les deux.

Il n'y a **pas** de champ séparé « code promo » et « code parrain ». L'utilisateur ne sait pas — et n'a pas à savoir — de quelle nature est le code qu'on lui a donné.

type de code	effet
REDUCTION	baisse le prix
AMBASSADEUR	baisse le prix et verse une commission à son propriétaire
PARRAINAGE	verse une commission à son propriétaire (généré à l'inscription)

Un code invalide ne fait jamais échouer la souscription. Elle réussit sans remise ni commission.

Authentification. Jeton JWT requis. **Scope pays** habituel.

2. Vérifier un code avant de payer — `POST /codes/valider`

À appeler quand l'utilisateur finit de saisir, pour afficher le prix barré.

```
{ "code": "amba247", "plan_uuid": "574612e9-...", "pays": "benin" }
```

La casse et les espaces sont normalisés : `amba247` et `AMBA247` désignent le même code.

Code accepté — 201

```
{
  "valide": true,
  "code": { "uuid": "9172b4e6-...", "code": "AMBA247", "type": "AMBASSADEUR", "libelle": null, "proprietaire_id": 26757 },
  "remise": { "type": "POURCENTAGE", "valeur": 10, "montant_remise": 200, "prix_initial": 2000, "prix_final": 1800 }
}
```

Affichez `prix_initial` barré et `prix_final` en évidence. **N'appliquez pas le pourcentage vous-même** : l'arrondi et le plafonnement sont faits côté serveur, et un calcul local finirait par diverger d'un franc.

Un code de parrainage pur est **valide sans objet** `remise` — il ne réduit rien, il désigne un bénéficiaire de commission. Ne traitez pas son absence comme une erreur.

Code refusé — 201 aussi

```
{ "valide": false, "motif": "INTROUVABLE" }
```

Ce n'est pas une erreur HTTP. Un code refusé est une réponse normale : testez `valide`, pas le statut.

motif	message suggéré
INTROUVABLE	Ce code n'existe pas.
INACTIF	Ce code n'est plus valable.
EXPIRE	Ce code a expiré.
PAS_ENCORE_VALIDE	Ce code n'est pas encore utilisable.
QUOTA_TOTAL_ATTEINT	Ce code a atteint son nombre d'utilisations.
DEJA_UTILISE	Vous avez déjà utilisé ce code.
PLAN_NON_ELIGIBLE	Ce code ne s'applique pas à cette formule.
AUTO_UTILISATION	Vous ne pouvez pas utiliser votre propre code.

QUOTA_TOTAL_ATTEINT et DEJA_UTILISE se ressemblent mais n'appellent pas le même message : le premier veut dire « trop tard », le second « déjà servi ».

3. Souscrire avec un code — POST /abonnements/souscrire

```
{ "plan_uuid": "574612e9-...", "code": "AMBA247", "pays": "benin" }
```

Le champ s'appelle `code`. `code_parrainage` reste accepté comme alias pour les versions déjà déployées, mais n'accepte que les codes de parrainage dans l'esprit ; utilisez `code`.

Réponse — 201

```
{
  "uuid": "abo-...", "statut": "EN_ATTENTE",
  "code_id": 52938, "montant_remise": 200,
  "parrain_id": 26757, "montant_paye": 0
}
```

champ	interprétation
code_id non nul	le code a été retenu et sa place consommée
code_id: null	aucun code retenu — absent, invalide, ou place prise entre-temps
montant_remise	remise obtenue, en devise du plan
parrain_id	bénéficiaire de la commission, s'il y en a un

⚠ **La place peut avoir été prise entre l'aperçu et l'achat.** Un code limité à 100 personnes peut atteindre son plafond pendant que l'utilisateur remplit le formulaire. Le serveur souscrit alors **sans le code** plutôt que d'échouer le paiement. Comparez donc `montant_remise` à ce que l'aperçu annonçait avant d'afficher le prix final — c'est le seul moment où l'écart se voit.

`montant_paye` vaut 0 tant que l'abonnement n'est pas encaissé ; le prix à régler est `prix du plan - montant_remise`.

4. Ce que l'écran doit faire

Un seul champ, bien libellé. « Code promo ou parrainage » plutôt que « Code promo » : l'utilisateur ne sait pas ce qu'il a en main.

Valider à la saisie, pas au paiement. Un code refusé découvert après le tunnel de paiement est une mauvaise surprise ; `POST /codes/valider` sert exactement à l'éviter.

Afficher le prix barré. `prix_initial` → `prix_final` rend la remise tangible. Sans cela, l'utilisateur doute que son code ait servi.

Revérifier après la souscription. Si `montant_remise` est inférieur à l'aperçu, dites-le : « Ce code n'était plus disponible, votre abonnement se poursuit au prix normal. »

Ne pas bloquer sur un code refusé. Le champ est facultatif ; laissez toujours la souscription possible.

5. Exemple Flutter

```
class CodesApi {
  CodesApi(this._client, {required this.baseUrl, required this.token, required this.pays});
  final http.Client _client;
  final String baseUrl, token, pays;

  Future<ResultatCode> valider(String code, String planUuid) async {
    final r = await _client.post(
      Uri.parse('$baseUrl/codes/valider'),
      headers: {'Authorization': 'Bearer $token', 'Content-Type': 'application/json'},
      body: jsonEncode({'code': code.trim(), 'plan_uuid': planUuid, 'pays': pays}),
    );
    // Un code refusé revient en 201 : c'est `valide` qui tranche, pas le statut.
    return ResultatCode.fromJson(jsonDecode(r.body));
  }
}

class ResultatCode {
  const ResultatCode({required this.valide, this.motif, this.remise});
  final bool valide;
  final String? motif;
  /// `null` sur un code de parrainage : il ne réduit rien, il désigne un
  /// bénéficiaire de commission. Ce n'est pas une erreur.
  final Remise? remise;

  String get message => switch (motif) {
    'INTROUVABLE' => 'Ce code n'existe pas.',
    'INACTIF' => 'Ce code n'est plus valable.',
    'EXPIRE' => 'Ce code a expiré.',
    'PAS_ENCORE_VALIDE' => 'Ce code n'est pas encore utilisable.',
    'QUOTA_TOTAL_ATTEINT' => 'Ce code a atteint son nombre d'utilisations.',
    'DEJA_UTILISE' => 'Vous avez déjà utilisé ce code.',
    'PLAN_NON_ELIGIBLE' => 'Ce code ne s'applique pas à cette formule.',
    'AUTO_UTILISATION' => 'Vous ne pouvez pas utiliser votre propre code.',
    _ => 'Code non pris en compte.',
  };
}
```

Souscrire, puis vérifier ce qui a réellement été appliqué

```
final apercu = await codes.valider(champCode.text, plan.uuid);
final abonnement = await abonnements.souscrire(plan.uuid, code: champCode.text);

// La place a pu partir pendant que l'utilisateur remplissait le formulaire.
final attendu = apercu.remise?.montantRemise ?? 0;
if (attendu > 0 && abonnement.montantRemise < attendu) {
  afficherInfo('Ce code n'était plus disponible. Votre abonnement se poursuit au prix normal.');
```

```
}

final aRegler = plan.prix - abonnement.montantRemise;
```

6. Rappels

Un seul champ pour tous les codes. Réduction, ambassadeur, parrainage : le registre tranche, pas l'application.

Un code refusé revient en 201. Testez `valide`, jamais le statut HTTP.

Un code valide sans `remise` est normal — c'est un code de parrainage.

Ne recalculez pas la remise. Arrondi et plafonnement sont faits côté serveur ; une remise de 100 % rend l'abonnement gratuit, jamais un prix négatif.

Vérifiez `montant_remise` après la souscription. L'aperçu peut dater : c'est là que se voit une place prise entre-temps.

La casse et les espaces sont ignorés. Inutile de forcer les majuscules à la saisie — mais ça reste plus lisible à l'écran.